DISTRIBUTED SYSTEMS · FA–1 + FA–2 MINI PROJECT

Design and Implementation of a Fault-Tolerant Distributed Search Engine

Partitioned indexing · coordinator election · vector-clock ordering

Atharv Rao
PRN: 123B1B024

Parth Darandale
PRN: 123B1B040

01 · THE PROBLEM SPACE

How do we find one useful page in the vast web?



The web contains an enormous number of pages, articles, tutorials, databases, and media files. They are created by different organizations and hosted across many servers around the world.

When a person needs information, they cannot know where the correct page is stored. The technology that brings this scattered information together is a **search engine**.

> **Story:** You type “`how do Python data structures work?`”. A search engine discovers likely pages, understands their text, finds relevant candidates, ranks them, and gives you useful links in milliseconds.

Billions of scattered web pages

↓

User question

↓

Search engine

↓

Ranked links and answers

Crawl
Collect public pages

Parse
Extract usable text

Index
Organize terms and documents

Retrieve & rank
Return the best results

02 · WHY DISTRIBUTED SYSTEMS?

Search at internet scale cannot depend on one machine

8.5+ billion
searches estimated per day worldwide

Millions
of pages must be indexed and refreshed

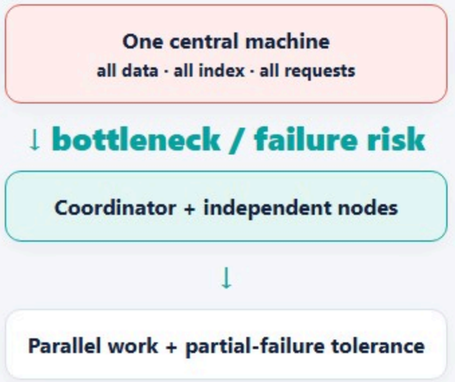
Milliseconds
expected time to return useful results

What is a distributed system?

A distributed system is a collection of independent computers or processes that communicate over a network and cooperate to appear as one service to users.

Why it is needed for search

- Split a large document corpus and index across nodes.
- Search partitions concurrently instead of sequentially.
- Serve results even if one node fails.
- Scale horizontally by adding nodes.



Our design: the coordinator does not contain all postings. Each node owns one document shard and its local inverted index; the coordinator only orchestrates and merges results.

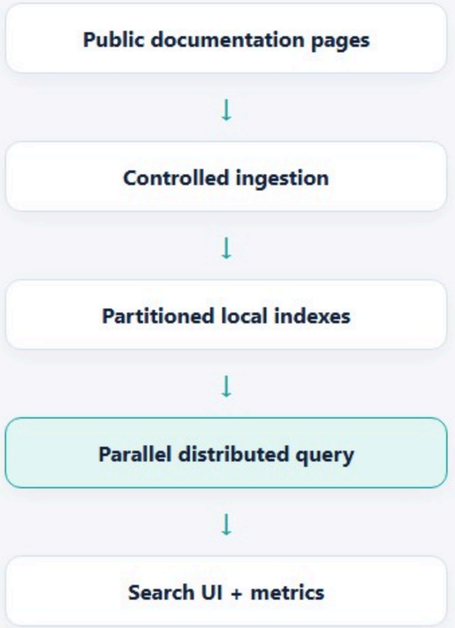
03 · OUR PROJECT

What are we building?

We are building a small but real distributed search engine for computer-science and technology documentation.

The project demonstrates the same essential journey used by large search systems: retain raw web pages, create searchable documents, construct indexes, search multiple partitions in parallel, and merge the most relevant answers.

- **Offline pipeline:** crawler → raw HTML → parser → logical documents → shards → local indexes.
- **Online pipeline:** browser query → coordinator → independent search nodes → global ranked results.
- **Demonstration objective:** show that the system remains useful even when one search node is unavailable.



04 · THE COMPLETE STORY

One page, one query, one distributed answer

Follow one documentation page from the public web until it becomes a ranked result.

A

Discover

Crawler downloads an allowed HTML page.

`000019.html`

↓

B

Understand

Parser creates a clean logical document.

`D19: Data Structures`

↓

C

Organize

Partitioner assigns it to one shard.

`Shard 2 → Node 2`

↓

D

Retrieve

Query reaches all nodes in parallel.

`"python data structures"`

↓

E

Rank

Coordinator merges the best candidates.

`Top results`

Story split: ingestion and indexing are offline preparation. Nodes and coordinator deliver the online distributed search.

05 · INGESTION

Discover, preserve, and explain the source

Controlled crawler pipeline



Key point: the original HTML is retained so parser improvements can be applied again without re-crawling the web.

06 · LOGICAL DOCUMENTS

Make web pages searchable

Before: noisy HTML

```
<nav>menu...</nav>
<h1>Data Structures</h1>
<p>Python includes lists...</p>
<script>analytics()</script>
```

Navigation, scripts and styles are not useful search content.

After: logical document

```
{
 "document_id": 19,
 "title": "Data Structures",
 "url": "https://...",
 "content": "Python includes lists..."
}
```

Title is displayed, URL opens the source, and content is indexed.

Title

Clickable result heading.

URL

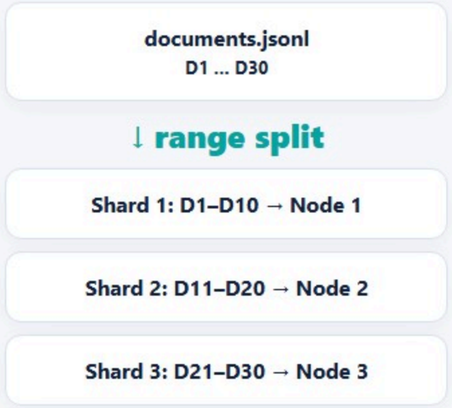
Returns users to source.

Content

Used for index and score.

Partition, tokenize, and index

Partitioning



Every document has exactly one owner node.

Inverted index example

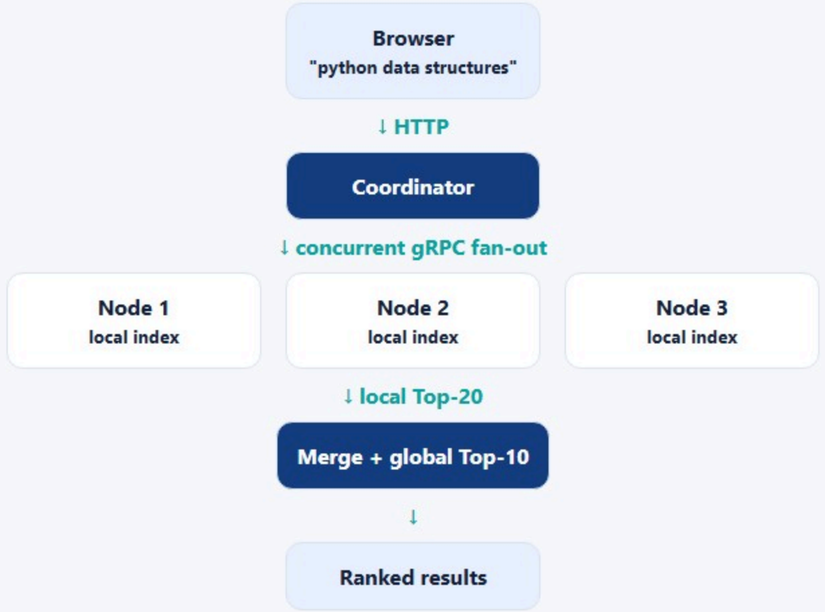
```
Text: "python data structures"
Tokens: [python, data, structures]

data → D19, tf=7,
positions=[1,18,…]
```

Nodes look up terms in local postings rather than scanning every page.

08 · DISTRIBUTED QUERY

Where distributed systems enter the search



Distributed principle: each node keeps its own postings and does local scoring; only concise Top-K candidates travel back to the coordinator.

Worked TF-IDF vectors and cosine scores

Vocabulary defines the vector dimensions: `V = [good, boy, girl, day, today]`. Every query and document has five coordinates in this example.

Document	Sentence	Full TF-IDF vector in V order
D1	good boy	`[0, 0.55, 0, 0, 0]`
D2	good girl	`[0, 0, 0.55, 0, 0]`
D3	good day today	`[0, 0, 0, 0.37, 0.37]`

1. Calculate global IDF

```
N = 3
df(good)=3  → IDF(good)=log(3/3)=0
df(boy)=1   → IDF(boy)=log(3)=1.10
df(girl)=1  → IDF(girl)=1.10
df(day)=1   → IDF(day)=1.10
df(today)=1 → IDF(today)=1.10
```

good is present in every document, so its IDF is zero. It contributes no discriminating weight.

2. Sample query: “good boy girl”

```
Query tokens = 3
TFq(good)=TFq(boy)=TFq(girl)=1/3

Q = [0, 0.37, 0.37, 0, 0]

Example: D1 has TF(boy)=1/2
0.5 x 1.10 = 0.55
```

The query and documents use the same five-dimensional vocabulary order.

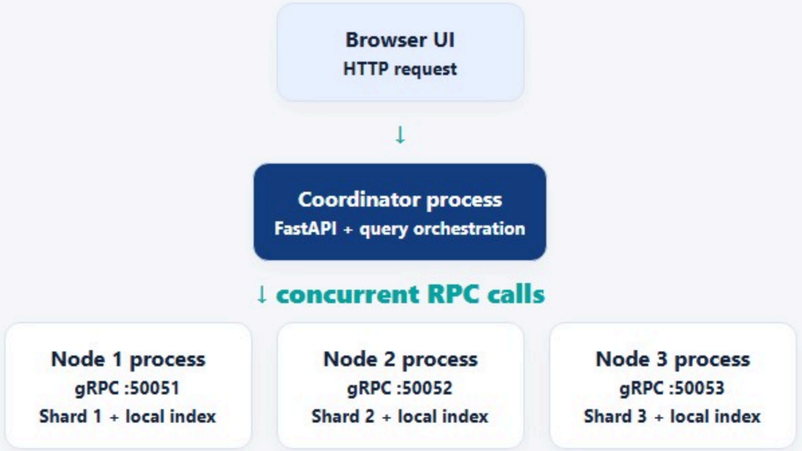
3. Score against D1
`cos(Q,D1) = 0.707`
Shares the term boy.

Score against D2
`cos(Q,D2) = 0.707`
Shares the term `girl`.

Score against D3
`cos(Q,D3) = 0.000`
Shares no informative query term.

Distributed meaning: local nodes calculate document TF from their own shards, but all nodes use the same global IDF vocabulary. Therefore their cosine scores are on one comparable scale, and the coordinator can merge them correctly.

How is the distributed infrastructure implemented?



Implementation: the coordinator and each node run as separate Python processes. Each node loads only its own shard-local inverted index and independently performs search work.

How RPC and gRPC are used in our project

What is RPC?

Remote Procedure Call lets one program call a function on another process or machine as if it were a local function. The communication details are handled by the RPC framework.



Why gRPC?

```
service SearchService {
 rpc Search(SearchRequest)
   returns (SearchResponse);
}

SearchRequest:
query, top_k

SearchResponse:
results, node_id,
statistics_version
```

Protocol Buffers define the exact typed messages. gRPC generates the client stub used by the coordinator and the server interface implemented by each node.

Coordinator role

Calls every node concurrently and enforces timeouts.

Node role

Calculates local TF-IDF cosine scores and returns Top-K.

Aggregation role

Merges comparable scores into the global ranking.

How the project applies Distributed Systems foundations

Unit 1 topic	Evidence in our project
Definition and goals	Independent node processes cooperate to provide one search experience with improved performance, scalability and availability.
Architecture	Coordinator-to-search-node architecture with document-partitioned local indexes.
Design issues	Partitioning, latency, globally comparable ranking, aggregation, and node failure are explicit design problems.
Middleware	gRPC provides the middleware layer for coordinator-to-node communication.
Distributed computation	Nodes run independently and exchange messages/RPC responses while the coordinator combines their work.

How the project applies Communication concepts

Unit 2 topic	Evidence in our project
Communication fundamentals	Configured node addresses: localhost:50051, 50052 and 50053.
Remote Procedure Call	`SearchService.Search` is invoked from coordinator to every node through gRPC.
Message-oriented communication	Protobuf messages carry query text, Top-K request, result list, node identity, and statistics version.
Parallel communication	Coordinator contacts all search nodes concurrently instead of waiting node by node.
Fault tolerance	Timeouts mark unavailable nodes; successful shards still provide partial results.

How to demonstrate the distributed behaviour

1

Start one node

Search returns results from its one local shard. The UI displays 1/3 healthy nodes.

2

Start all nodes

Query fan-out reaches every shard in parallel; the coordinator merges more candidates into global Top-K.

3

Stop one node

The coordinator marks it unavailable, but retains results from healthy nodes and reports degraded status.

C3	1	v1	6/6	3s / 9s
LEADER	EPOCH	STATISTICS VERSION	PROCESSES UP	BEACON / TIMEOUT

Processes

Start all | Stop all | Reset demo data

C1 — STANDBY

epoch **1** statistics **v1** N **31**

docs per node **N1=11 N2=10 N3=10**

beacons received **9** from **C3**

last beacon 2s ago (suspects leader at 9s)

`vector [31, 29, 142, 34, 31, 31]`

coordinator — Kill

C2 — STANDBY

epoch **1** statistics **v1** N **31**

docs per node **N1=11 N2=10 N3=10**

beacons received **9** from **C3**

last beacon 2s ago (suspects leader at 9s)

`vector [29, 31, 143, 34, 31, 31]`

coordinator — Kill

C3 — LEADER

epoch **1** statistics **v1** N **31**

docs per node **N1=11 N2=10 N3=10**

beacons sent **9**

node beacon replies: N1 ✓ N2 ✓ N3 ✓

`vector [31, 31, 151, 36, 33, 33]`

coordinator — Kill

N1 — FOLLOWER

epoch **1** statistics **v1** shard docs **11**

beacons received **9** from **C3**

last beacon 2s ago (suspects leader at 9s)

`vector [29, 29, 144, 36, 31, 31]`

node — Kill

N2 — FOLLOWER

epoch **1** statistics **v1** shard docs **10**

beacons received **9** from **C3**

last beacon 2s ago (suspects leader at 9s)

`vector [29, 29, 145, 34, 33, 31]`

node — Kill

N3 — FOLLOWER

epoch **1** statistics **v1** shard docs **10**

beacons received **9** from **C3**

last beacon 2s ago (suspects leader at 9s)

`vector [29, 29, 146, 34, 31, 33]`

node — Kill

Kill a coordinator or node to test failure handling. Vectors are written in the order [C1, C2, C3, N1, N2, N3]. Beacons arrive every 3 s; a process that hears nothing for 9 s suspects the leader.

15 · FA–2 · UNIT 3 + UNIT 4

From one coordinator to a coordinated, self-healing cluster

FA–1 gave us partitioned, parallel search behind a single coordinator. FA–2 removes that single point of failure and moves the system closer to how real distributed services are run.

UNIT 3 · Synchronization and coordination

Beacon (heartbeat) protocol · Bully election with epochs · vector clocks that order every message.

UNIT 4 · Real-world distributed systems

Replicated state and consistency trade-offs · web-based architecture · proposed AWS deployment with EC2 and S3.

Live demonstration

Kill the leader, watch the election, read the vector clocks, and add documents while the cluster heals itself.

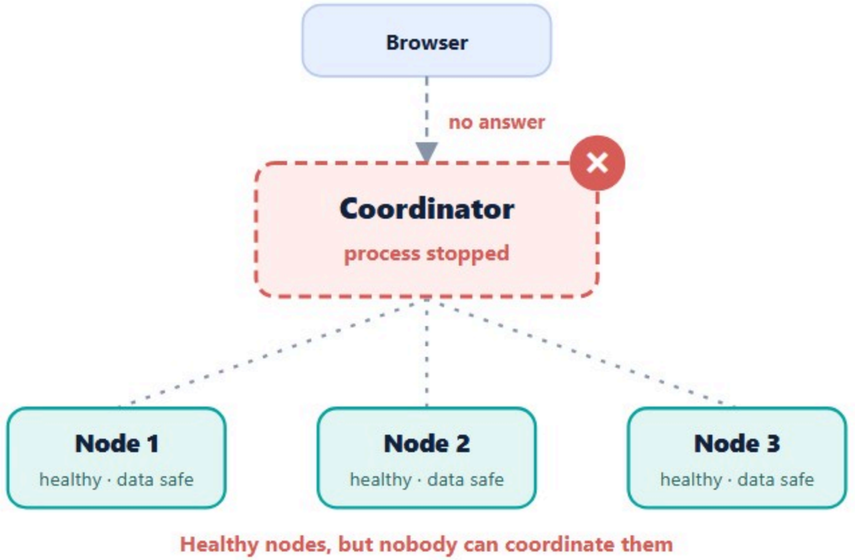
3
coordinators, one leader

6
processes with a vector clock

9 s
of silence before an election

21
automated tests, 11 end-to-end

Where the FA–1 architecture breaks



FA–1: one coordinator holds every connection. When it stops, three healthy nodes are cut off.

1 · Single point of failure
If the coordinator goes down there is no way to coordinate the nodes or to get data back from them. Search stops although every node is healthy.

2 · Only one coordinator
No standby, no failover target and no way to share coordination work.

3 · No way to add more coordinators
Nodes knew exactly one coordinator. With several: who is in charge, whose orders do the nodes obey, and how does a node find a replacement?

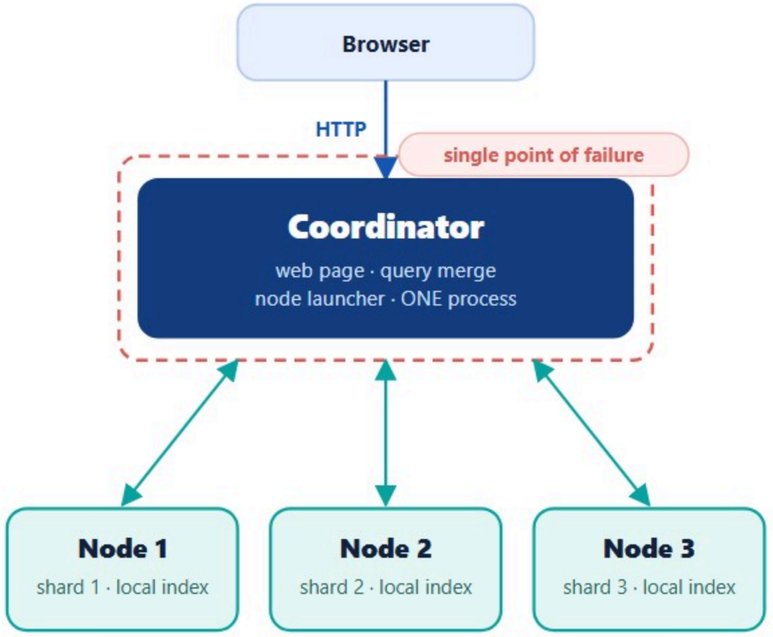
4 · Frozen data
The indexes were built once, before start-up. New documents could not be added while the system was running.

What the system therefore needs

several active coordinators · leader election · failure detection (beacons) · ordering of events (vector clocks) · replicated state

From one coordinator to a coordinator cluster

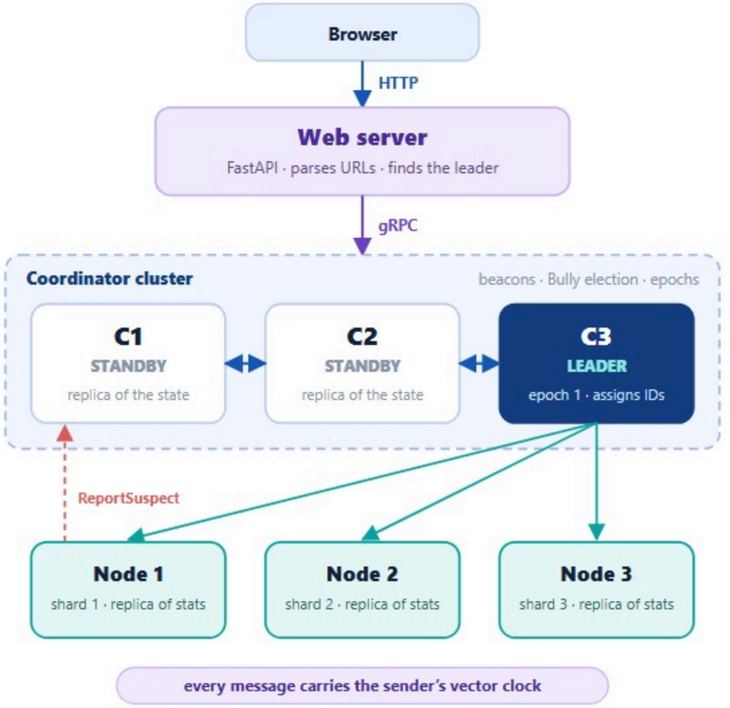
FA–1 · before



index built once, offline, before start-up

One process serves the page, merges results and launches the nodes. Lose it and everything stops.

FA–2 · after

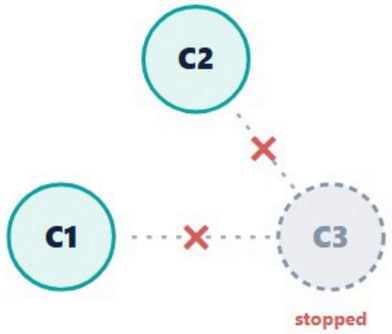


The web server parses URLs, serves the pages and finds the leader. Three coordinators run a Bully election: the leader writes, the standbys replicate and can take over. Nodes follow the leader of the highest epoch.

The Bully algorithm: the highest live ID wins

① The leader goes silent

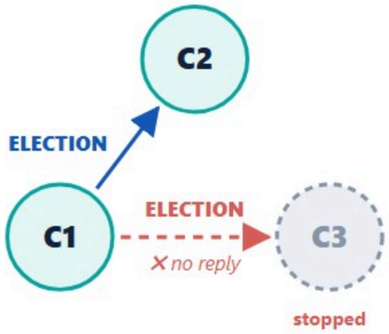
no beacon for 9 s



Nodes notice first and report;
C1 and C2 ping C3: no answer

② C1 starts an election

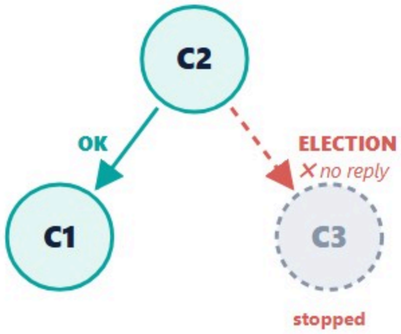
asks only higher IDs



IDs: C1 < C2 < C3
C3 does not answer

③ A higher ID takes over

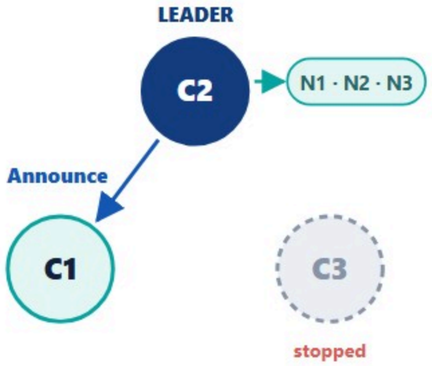
C2 answers OK, then asks C3



C1 stops and waits
for the winner

④ C2 wins: epoch 2

nobody higher answered



Announce (COORDINATOR) to C1 and
all nodes; beacons resume

ELECTION

Sent by a process that sees no leader, only to coordinators with a **higher** ID.

OK

A live, higher coordinator answers and takes over the election: it "bullies" the asker into waiting.

COORDINATOR (our `Announce`)

If nobody higher answers, the asker wins and announces itself to everyone, together with the new epoch.

Who notices, who verifies, who decides

1
Leader stops
C3 is killed from the admin console (or crashes). Its beacons stop.

2
Nodes notice
After 3 missed beacons (9 s) every node suspects the leader.

3
Nodes report
`ReportSuspect` goes to every coordinator.

4
Coordinators verify
A coordinator **pings** the leader. An answer means a false alarm.

5
Bully election
No answer: ELECTION goes to higher IDs. The highest live ID wins.

6
New epoch
C2 catches up on statistics, then announces **epoch 2** to all coordinators and nodes.

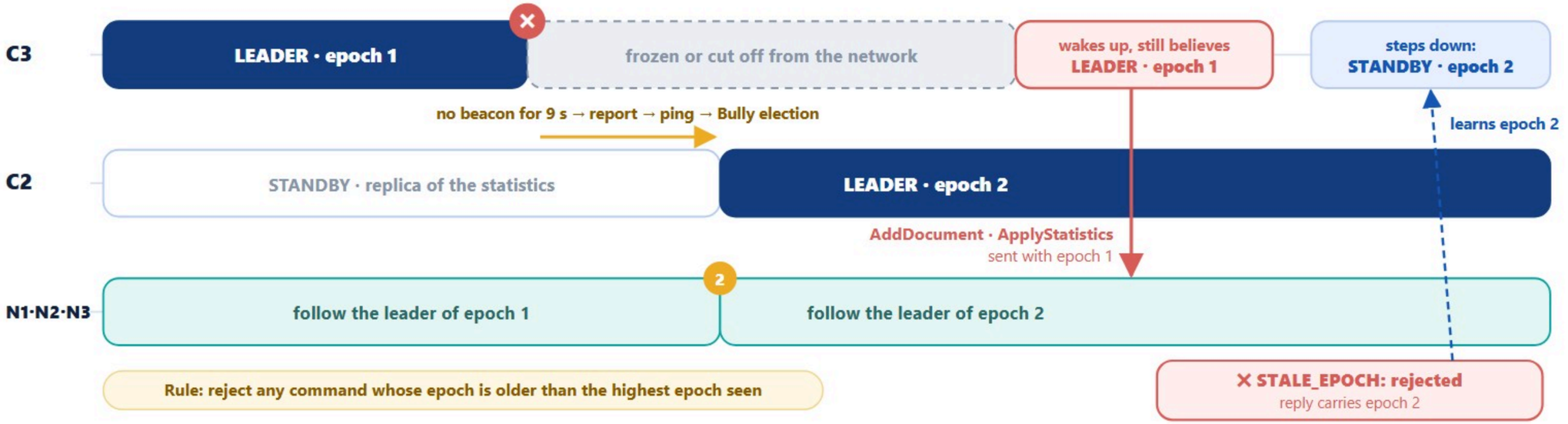
7
Service resumes
Beacons restart and the web server finds the new leader on its next poll.

Standbys watch too
Coordinators run the same silence timer, one beat later than the nodes, so a missing report cannot stall recovery.

Search keeps working
Any live coordinator can serve queries. Only adding documents needs a leader.

Revival
A returning C3 catches up first, then bullies back with a higher epoch (epoch 3). Tested with real processes.

Epochs: a returning old leader cannot give orders



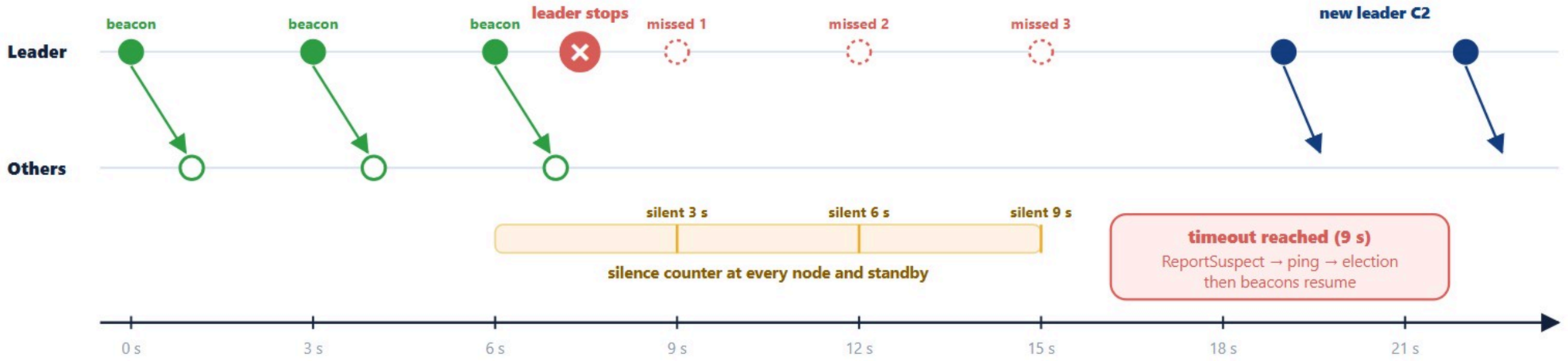
The rule

Every command carries the sender's **epoch** (the leader's term number). A process remembers the highest epoch it has seen and rejects anything older with `STALE_EPOCH`. The reply carries the new epoch, so the old leader steps down by itself.

Why it matters

Without epochs, a paused or cut-off leader could wake up and assign a document ID twice, or push outdated statistics. The epoch fences it off without having to kill it. Verified by a test: nodes reject epoch-1 commands once epoch 2 is announced.

“I am alive” every 3 seconds



interval 3 s · 3 missed beacons → timeout 9 s

Who sends

Only the leader: to both standby coordinators and all three nodes.

What it carries

Leader ID, epoch and the committed statistics version. The reply carries the receiver’s version, so a lagging process is caught up.

Why three misses

One lost or late beacon must not trigger an election. After 9 s of silence the ping check gives a second opinion.

Visible in the UI

Beacons sent / received counters and a countdown bar per process show the 9 s timer running.

Why logical time? Which event happened first?

Coordinators and nodes are separate processes with separate clocks. Wall-clock timestamps cannot say whether the leader's "assign ID" came before a node's "index document", or whether two nodes simply acted independently.

Lamport vs. vector: one Lamport number orders causally related events but cannot tell "concurrent" from "ordered". A vector can.

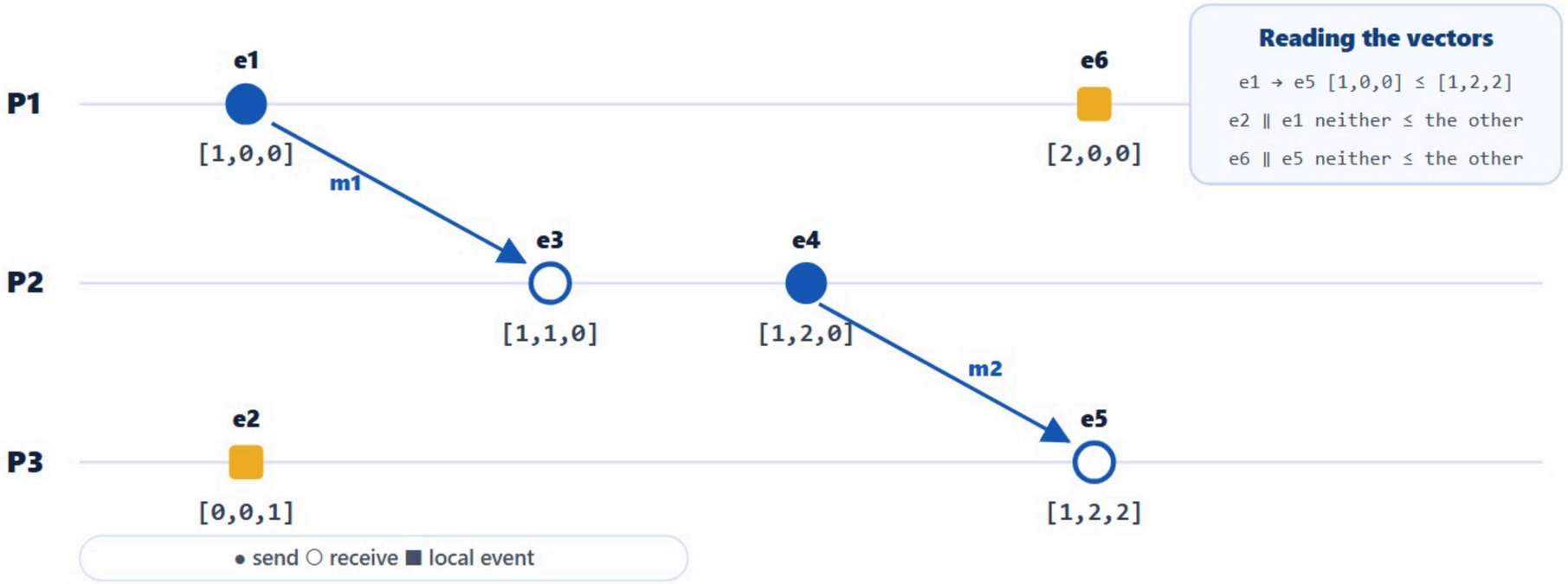
Vector clock rules (one counter per process)

① **local event or send**: own counter + 1; a send attaches the whole vector

② **receive**: take the element-wise **maximum** of own and received vector

③ then add 1 to the own counter

Compare: A → B if every entry of A ≤ B (and A ≠ B). If neither vector dominates, A ∥ B: the events are **concurrent**.



Worked example with three processes: e1 → e5 because [1,0,0] ≤ [1,2,2]; e2 ∥ e1 and e6 ∥ e5 are concurrent.

One vector in every message

Six processes, six counters, in the order `[C1, C2, C3, N1, N2, N3]`. The web server is an outside observer and has no clock.

```
message Stamp {
  map<string, uint64> clock = 1;  // full vector
  string msg_id = 2;               // send ↔ receive
  string op     = 3;               // operation tag
  string sender = 4;
}
```

Every gRPC request **and** reply carries a `Stamp` as an explicit field. A tracer ticks the clock on each send, receive and local event, and reports it to the admin console.

> **Also clocked:** beacons, elections, statistics updates and searches. The counters therefore grow over time, which is why vectors look like `[13, 19, 59, 11, 11, 11]` rather than `[0, 0, 1, 0, 0, 0]`.

#	PROCESS	EVENT	KIND	VECTOR [C1, C2, C3, N1, N2, N3]
1	**C3**	receives the document	local	[18, 18, 59, 11, 11, 11]
2	**C3**	assigns document ID 31	local	[18, 18, 60, 11, 11, 11]
3	**C3**	picks the least-loaded node (N1)	local	[18, 18, 61, 11, 11, 11]
4	**C3**	sends the document → N1	send	[18, 18, 62, 11, 11, 11]
5	**N1**	receives the document ← C3	receive	[18, 18, 62, 12, 11, 11]
	local [16, 16, 51, 11, 9, 9] max message [18, 18, 62, 11, 11, 11] = [18, 18, 62, 11, 11, 11] → +1 on N1 → [18, 18, 62, 12, 11, 11]			
6	**N1**	indexes it	local	[18, 18, 62, 13, 11, 11]
7	**N1**	confirms indexing → C3	send	[18, 18, 62, 14, 11, 11]
8	**C3**	receives the confirmation ← N1	receive	[18, 18, 63, 14, 11, 11]

Real trace of adding document 31 ("Event loop — Python 3.15.0 documentation"), captured from the running system. Pink: entries that changed. Underlined: the process's own counter.

What vector clocks give us

No synchronized clocks

Order comes from messages, not from wall-clock time, so clock drift between machines is irrelevant.

True causality

If A happened before B then $A \leq B$, and the reverse holds too: comparing two vectors tells us exactly whether they are related.

Detects concurrency

Two documents added at the same moment are shown as concurrent instead of being falsely ordered. This is the basis for spotting conflicting updates.

Explains failures

The causal trace shows what each process knew when it acted: which beacon was missed, which report started the election.

Reveals stale knowledge

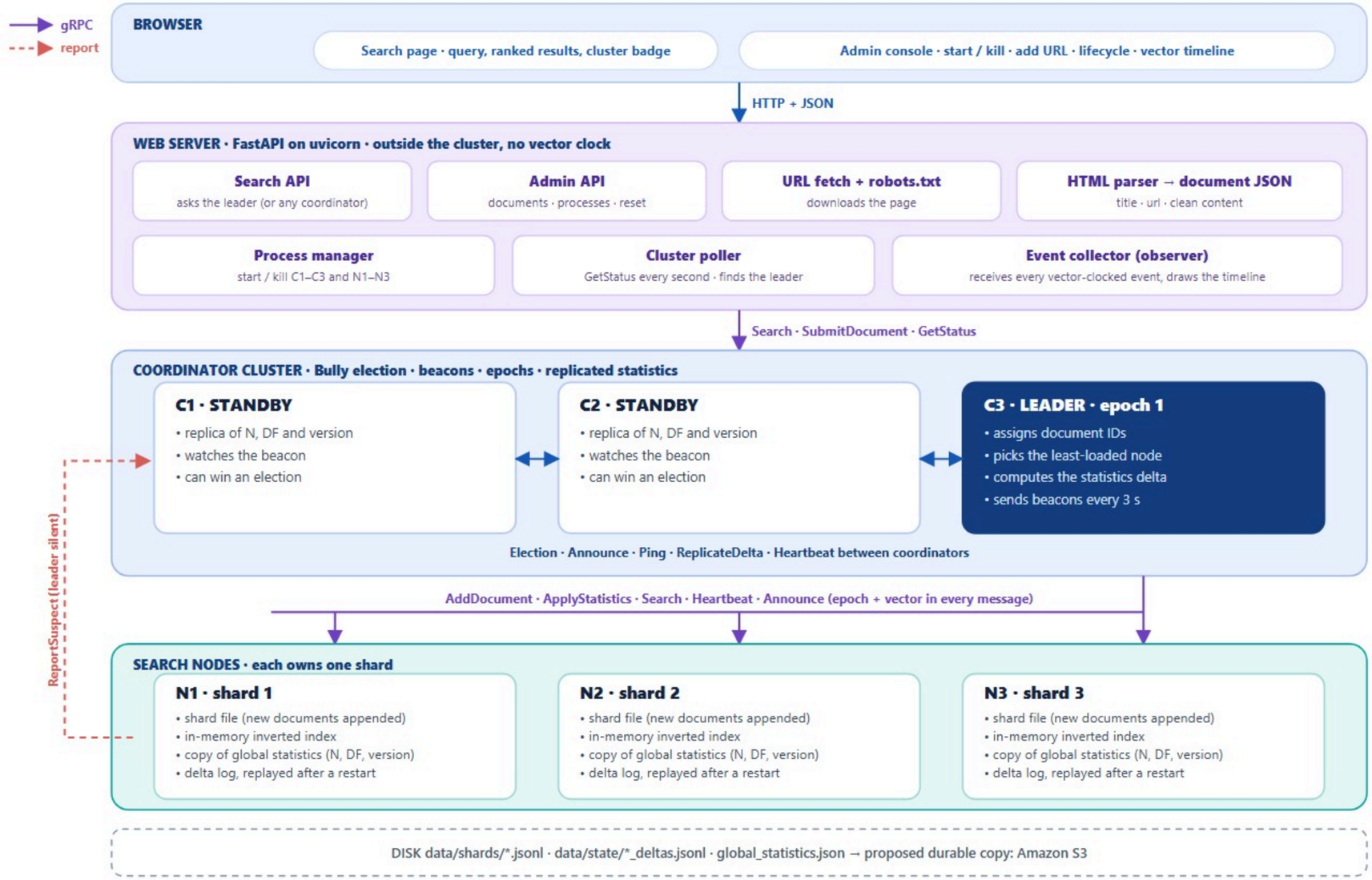
A process whose vector or epoch lags is behind and must catch up before it acts.

Survives restarts

Merging by element-wise maximum rebuilds a restarted process's knowledge from the first message it receives.

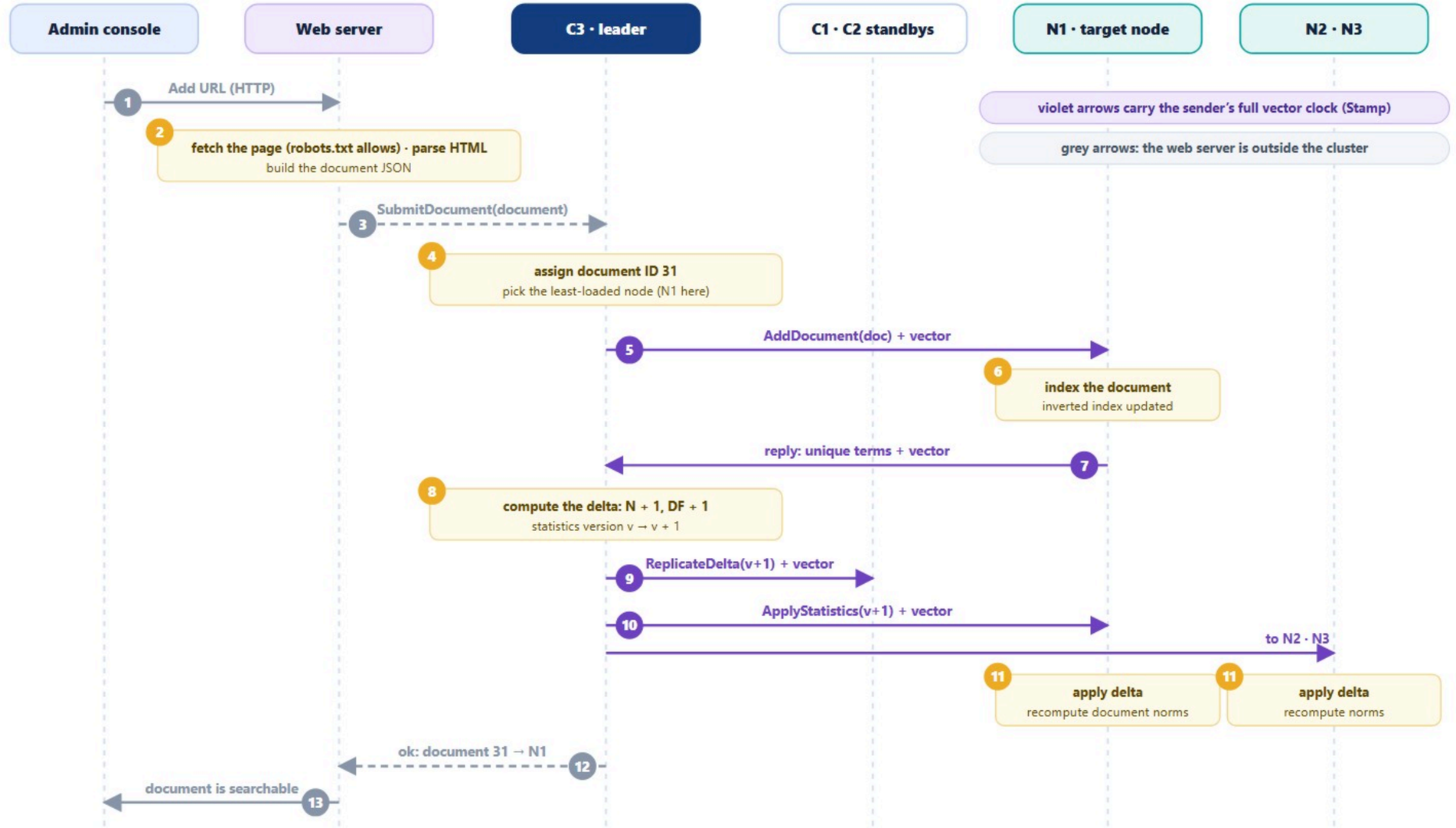
Costs and limits: the vector grows with the number of processes (6 entries here) and the counters grow with every beacon. Vector clocks order events and expose concurrency; they do not by themselves prevent deadlocks. We use them for ordering, conflict detection and diagnosis.

The complete system, top to bottom



Browser → web server → coordinator cluster → search nodes. Violet arrows are gRPC messages carrying epoch and vector clock; the red dashed arrow is a node reporting a silent leader.

One URL, end to end



The web server does the parsing; the leader assigns the ID, chooses the node and replicates the statistics change. Two URLs submitted together run steps 5–7 in parallel on different nodes.

Keeping every copy of the statistics in step

1

Compute the delta

Leader: N + 1 and +1 document frequency for every term of the new document. Version v → v + 1.

2

Replicate

Delta goes to the standby coordinators first, so a new leader always has it.

3

Push to nodes

All nodes apply deltas strictly in order. A gap triggers a resend.

4

Recompute norms

N changed, so every IDF changed: nodes rebuild all document vector norms.

5

Searchable

The committed version advances; searches ignore nodes on any other version.

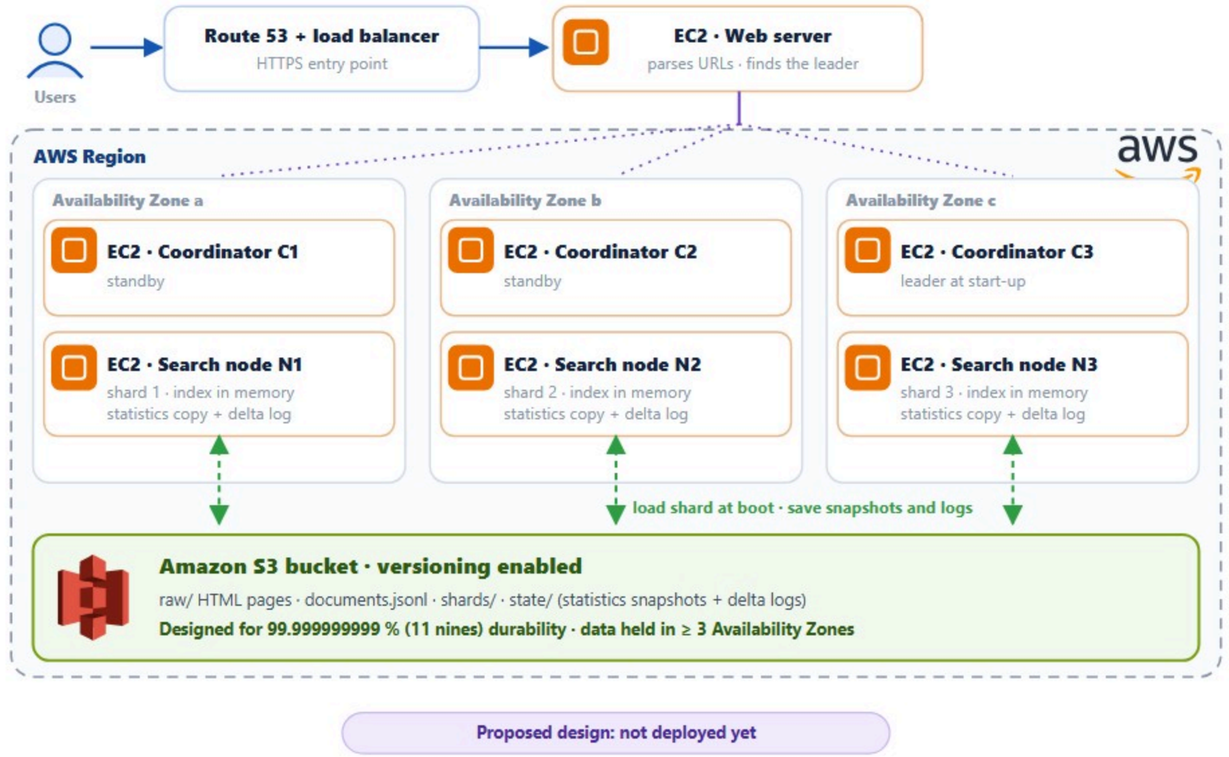
Replication, recovery and trade-offs

- Every coordinator and node keeps the base statistics plus an ordered **delta log** on disk.
- A restarted process replays its log; the leader's next beacon reveals its version and the missing deltas are pushed.
- **Availability first:** reads work from any coordinator and from the live nodes; consistency is restored by catch-up and protected by version checks and epochs.

Honest limits

- If the leader dies after a node stored a document but before the delta was replicated, that one document is not counted until it is added again.
- A search racing an update can briefly skip a node that is one version behind.
- Same idea as version stamps in GFS and HDFS: a replica with an old version is not trusted.

Running it on AWS: EC2 plus S3



Coordinators and nodes are spread over three Availability Zones. S3 holds the data that must survive any single machine.

1

An EC2 instance fails

Beacons stop; the cluster elects a new leader if it was a coordinator, and search continues on the healthy nodes.

2

A replacement is launched

A launch template starts a fresh EC2 instance; its start-up script copies the shard and the latest snapshot from S3.

3

It catches up and rejoins

The process replays its delta log, the leader's beacon shows its version, and the missing deltas are pushed.

Why S3: S3 Standard is designed for 99.999999999 % (11 nines) durability and stores objects across at least three Availability Zones, with 99.99 % designed availability. It is the safe copy; EC2 disks are disposable.

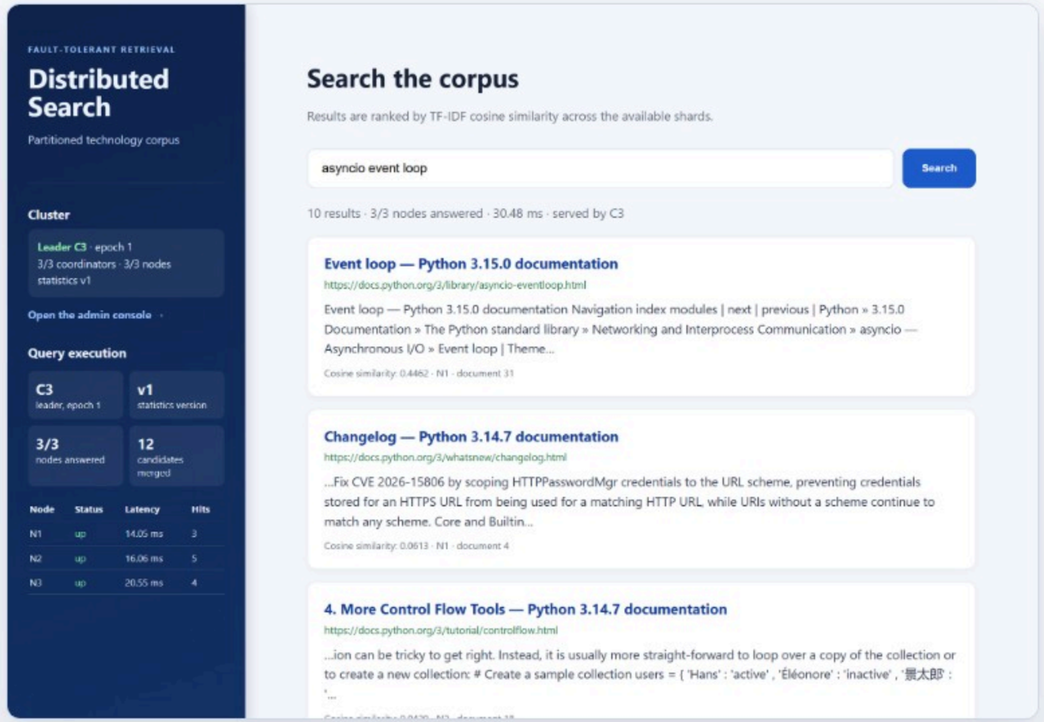
Applying Unit 3: synchronization and coordination

Unit 3 topic	Evidence in our project	Status
Logical clocks, vector algorithm	A `Stamp` with the full vector in every gRPC message; live causal timeline, event comparison (before / after / concurrent).	IMPLEMENTED
Election algorithms	Bully election among C1–C3, extended with epochs to fence off stale leaders.	IMPLEMENTED
Beacon protocol	Leader beacon every 3 s, suspicion after 9 s of silence, counters and countdown bars in the admin console.	IMPLEMENTED
Mutual exclusion	Only the elected leader accepts writes (single writer). A classic algorithm such as Ricart–Agrawala is not used.	LEADER-BASED
Global state	Per-process status and a causal trace of events. We do not take Chandy–Lamport snapshots.	NOT DONE

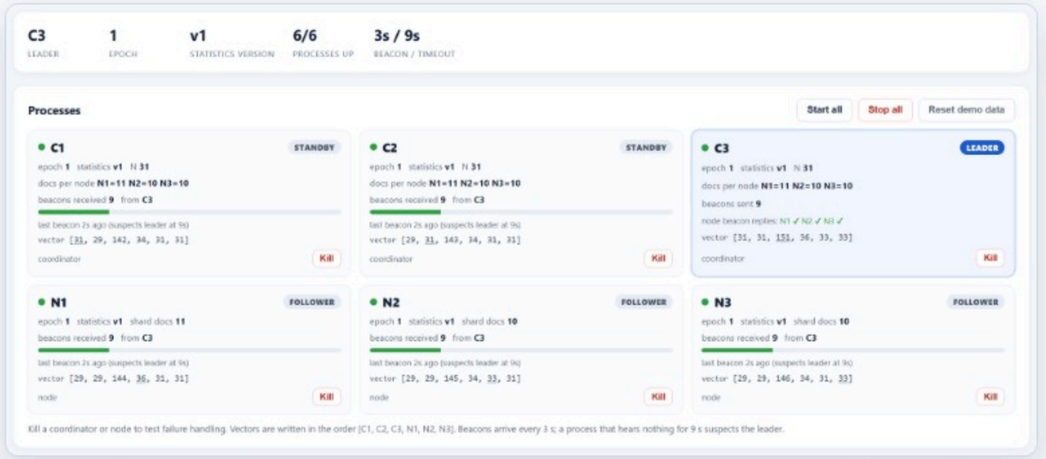
Applying Unit 4: real-world distributed systems

Unit 4 topic	Evidence in our project	Status
Distributed web-based systems	Browser → stateless web server → gRPC coordinators → nodes. The HTTP front end is separate from the coordination logic.	IMPLEMENTED
Replication and consistency trade-offs	Statistics deltas replicated to standbys and nodes, version checks on every search, epoch fencing; availability favoured over strict consistency.	IMPLEMENTED
Cloud case study: AWS	EC2 instances across three Availability Zones with S3 as durable store; failure recovery through replacement instances.	DESIGNED
Distributed storage ideas (GFS, HDFS)	S3 plays the durable-storage role; version numbers identify stale replicas, as chunk versions do in GFS and generation stamps in HDFS.	CONCEPT
Kubernetes	Containerising coordinators and nodes (a StatefulSet for nodes) is the natural next deployment step.	FUTURE WORK

Two pages: search for users, admin for operators



Search page (/): served by C3, leader of epoch 1, statistics v1, 3/3 nodes answered. The page added from a live URL (document 31, node N1) ranks first.



Admin console (`/admin.html`): one card per process, with live role, epoch, beacon counters and vector clock; Start / Kill for every coordinator and node.

> **Node controls moved here.** The search page no longer starts or stops nodes; operators do that from the admin console.

Control and observe every process

C3	1	v1	6/6	3s / 9s
LEADER	EPOCH	STATISTICS VERSION	PROCESSES UP	BEACON / TIMEOUT

Processes

Start all | Stop all | Reset demo data

C1 STANDBY
epoch **1** statistics **v1** N **31**
docs per node **N1=11 N2=10 N3=10**
beacons received **9** from **C3**
last beacon 2s ago (suspects leader at 9s)
`vector [31, 29, 142, 34, 31, 31]`
coordinator — Kill

C2 STANDBY
epoch **1** statistics **v1** N **31**
docs per node **N1=11 N2=10 N3=10**
beacons received **9** from **C3**
last beacon 2s ago (suspects leader at 9s)
`vector [29, 31, 143, 34, 31, 31]`
coordinator — Kill

C3 LEADER
epoch **1** statistics **v1** N **31**
docs per node **N1=11 N2=10 N3=10**
beacons sent **9**
node beacon replies: N1 ✓ N2 ✓ N3 ✓
`vector [31, 31, 151, 36, 33, 33]`
coordinator — Kill

N1 FOLLOWER
epoch **1** statistics **v1** shard docs **11**
beacons received **9** from **C3**
last beacon 2s ago (suspects leader at 9s)
`vector [29, 29, 144, 36, 31, 31]`
node — Kill

N2 FOLLOWER
epoch **1** statistics **v1** shard docs **10**
beacons received **9** from **C3**
last beacon 2s ago (suspects leader at 9s)
`vector [29, 29, 145, 34, 33, 31]`
node — Kill

N3 FOLLOWER
epoch **1** statistics **v1** shard docs **10**
beacons received **9** from **C3**
last beacon 2s ago (suspects leader at 9s)
`vector [29, 29, 146, 34, 31, 33]`
node — Kill

Kill a coordinator or node to test failure handling. Vectors are written in the order [C1, C2, C3, N1, N2, N3]. Beacons arrive every 3 s; a process that hears nothing for 9 s suspects the leader.

Leader and epoch
The leader's card is highlighted; the summary shows leader, epoch, statistics version and processes up.

Beacon counters
The leader shows beacons sent and which nodes replied; everyone else shows beacons received and a bar counting down to the 9 s timeout.

Live vector clock
Every card shows that process's current vector, in the order C1 … N3, with its own entry underlined.

Start / Kill
Any coordinator or node can be killed and restarted to demonstrate failure handling.

Watching a URL become a searchable document

Lifecycle · Add · https://docs.python.org/3/library/asyncio-eventloop.html

+0.00s	web	**receives the add request** source: https://docs.python.org/3/library/asyncio-eventloop.html · kind: url outside the cluster: no vector clock
+0.00s	web	**fetches the page** url: https://docs.python.org/3/library/asyncio-eventloop.html outside the cluster: no vector clock
+0.79s	web	**page downloaded** bytes: 248963 · content type: text/html · fetch ms: 259 outside the cluster: no vector clock
+1.07s	web	**HTML parsed** title: Event loop — Python 3.15.0 documentation · characters: 60922 outside the cluster: no vector clock
+1.07s	web	**builds the document JSON** title: Event loop — Python 3.15.0 documentation · url: https://docs.python.org/3/library/asyncio-eventloop.html · content length: 60922 outside the cluster: no vector clock
+1.07s	web	**sends the document to the leader** leader: C3 outside the cluster: no vector clock
+1.09s	C3	**receives the document** title: Event loop — Python 3.15.0 documentation · url: https://docs.python.org/3/library/asyncio-eventloop.html · content length: 60922 `[18, 18, 59, 11, 11, 11]`
+1.09s	C3	**assigns the document ID** doc id: 31 `[18, 18, 60, 11, 11, 11]`
+1.09s	C3	**picks the least-loaded node** node: N1 · load: {"N1":11,"N2":10,"N3":10}

Lifecycle panel: the web server's steps (no vector clock) come first, then the leader's steps with the vector clock on every line.

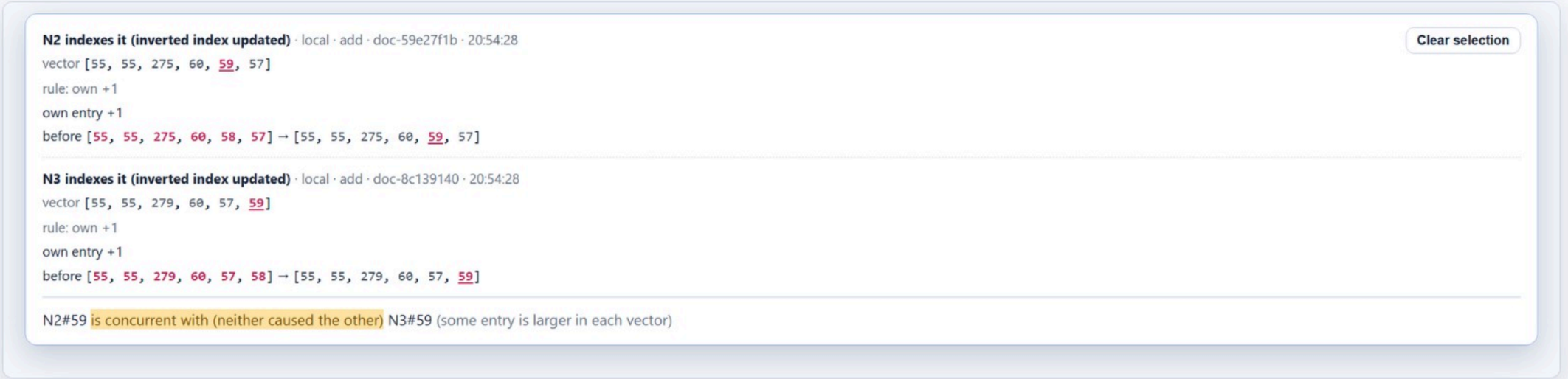
Document JSON built by the web server

```
{
  "title": "Event loop — Python 3.15.0 documentation",
  "url": "https://docs.python.org/3/library/asyncio-eventloop.html",
  "source": "admin",
  "content": "…clean visible text…",
  "content_length": …
}
document_id is added by the leader: 31
```

Then: the leader routes it to the least-loaded node (N1), computes the statistics delta, replicates it to C1 and C2, and pushes it to N1–N3. Search finds it immediately.

Two documents at once: concurrent, not ordered

"Add 2 sample texts at once" sends both documents to different nodes in parallel. Neither indexing event caused the other, and the vector clocks prove it.



Inspector of the admin console: N2's event `[55, 55, 275, 60, 59, 57]` and N3's event `[55, 55, 279, 60, 57, 59]`. N2 is ahead in N2; N3 is ahead in C3, N3. Neither vector dominates, so the events are **concurrent**.

> **Why a single Lamport number is not enough:** sorting by it (with a tie-break) always produces an order, even between events that were independent. Vectors do not pretend.

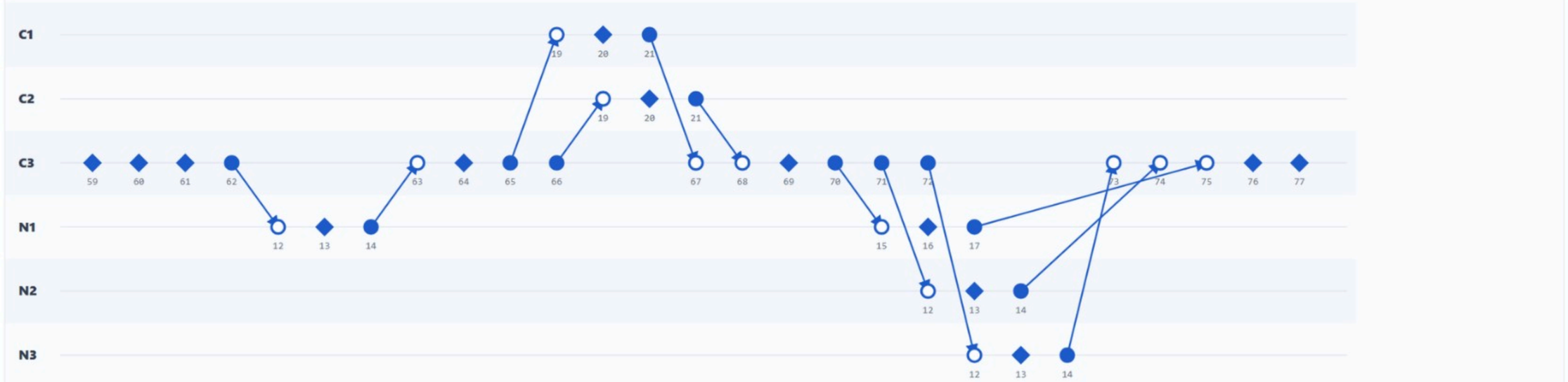
The whole add as a causal diagram

Vector-clock timeline

20:53:33 Add · https://docs.python.org/3/l | Latest add | Latest election | Latest search | Compare last two adds | Beacons | ▶ Replay | normal | show full vectors on the dots

Each dot is an event on one process. Dots show that process's own counter. Arrows are messages: the receiver takes the element-wise maximum of both vectors, then adds one.

send · receive · local event | add · election · search · sync · beacon



How to read it

One lane per process, one dot per event: ● send, ○ receive, ◆ local. Arrows are messages; the number under a dot is that process's own counter.

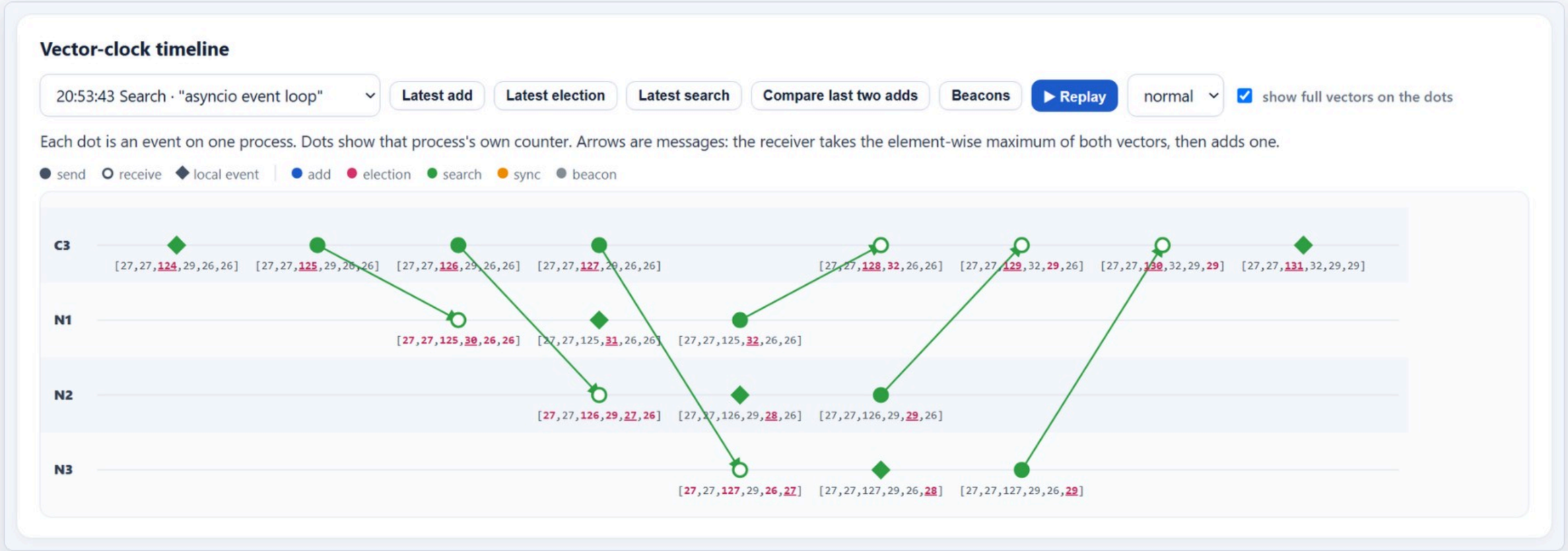
What you see

C3 sends the document to N1, hears back, then replicates to C1 and C2 and pushes the new statistics to all three nodes.

Replay

The console can animate the diagram step by step, which is useful on stage.

Zoom in: the whole vector on every event



One search (C3 → N1, N2, N3 and back). Only the processes that took part get a lane. Pink entries changed at that event; the underlined entry is the process's own counter.

Reading one arrow: when a node receives the query it takes the entry-wise maximum of its own vector and the leader's, then adds one to its own entry. That is why its vector suddenly "learns" the leader's counter.

Every event, its rule and its vector

Vector trace `[C1, C2, C3, N1, N2, N3]`

☑ show the max-merge arithmetic for receives

Each row is one event, in causal order. A vector has one counter per process, in the order shown above, for example `[0, 0, 1, 0, 0, 0]` means "C3 has done one event and nobody else has done anything yet". **Pink** entries changed at this event; the underlined entry is the process's own counter. Click a row (or a dot) to inspect it, click a second one to compare.

STEP	PROCESS	EVENT	MESSAGE	RULE	VECTOR CLOCK `[C1, C2, C3, N1, N2, N3]`	DETAILS
1	C3	• **local** receives the document		own +1	`[18, 18, 59, 11, 11, 11]`	title: Event loop — Python 3.15.0 documentation · url: https://docs.python.org/3/library/asyncio-eventloop.html · content length: 60922
2	C3	• **local** assigns the document ID		own +1	`[18, 18, 60, 11, 11, 11]`	doc id: 31
3	C3	• **local** picks the least-loaded node		own +1	`[18, 18, 61, 11, 11, 11]`	node: N1 · load: {"N1":11,"N2":10,"N3":10}
4	C3	→ **send** sends the document to a node	→ N1 5c0c079151	own +1, attach vector	`[18, 18, 62, 11, 11, 11]`	
5	N1	← **receive** receives the document	← C3 5c0c079151	max(local, message), then own +1	`[18, 18, 62, 12, 11, 11]`	doc id: 31
	local `[16, 16, 51, 11, 9, 9]` **max** message from C3 `[18, 18, 62, 11, 11, 11]` = `[18, 18, 62, 11, 11, 11]` **then +1 on N1** → `[18, 18, 62, 12, 11, 11]`					
6	N1	• **local** indexes it (inverted index updated)		own +1	`[18, 18, 62, 13, 11, 11]`	doc id: 31 · tokens: 9113 · unique terms: 1132 · shard documents: 11
7	N1	→ **send** confirms indexing	→ C3 5c0c079151:reply	own +1, attach vector	`[18, 18, 62, 14, 11, 11]`	
8	C3	← **receive** receives the indexing confirmation	← N1 5c0c079151:reply	max(local, message), then own +1	`[18, 18, 63, 14, 11, 11]`	
	local `[18, 18, 62, 11, 11, 11]` **max** message from N1 `[18, 18, 62, 14, 11, 11]` = `[18, 18, 62, 14, 11, 11]` **then +1 on C3** → `[18, 18, 63, 14, 11, 11]`					
9	C3	• **local** computes the new global statistics (N, DF)		own +1	`[18, 18, 64, 14, 11, 11]`	version: 1 · total docs: 31 · new terms: 1132
10	C3	→ **send** replicates the delta to a standby	→ C1 02782bfc2b	own +1, attach vector	`[18, 18, 65, 14, 11, 11]`	
11	C1	← **receive** receives the replicated delta	← C3 02782bfc2b	max(local, message), then own +1	`[19, 18, 65, 14, 11, 11]`	versions: [1]
	local `[18, 16, 49, 9, 9, 9]` **max** message from C3 `[18, 18, 65, 14, 11, 11]` = `[18, 18, 65, 14, 11, 11]` **then +1 on C1** → `[19, 18, 65, 14, 11, 11]`					
11	C3	→ **send** replicates the delta to a standby	→ C2	own +1, attach vector	`[18, 18, 66, 14, 11, 11]`	

Check it by hand: local `[16, 16, 51, 11, 9, 9]` **max** message `[18, 18, 62, 11, 11, 11]` = `[18, 18, 62, 11, 11, 11]`, then +1 on N1 gives `[18, 18, 62, 12, 11, 11]`. Rules: **local** own +1 · **send** own +1 and attach the vector · **receive** max, then own +1.

Kill the leader, watch the election

C2	2	v3	5/6	3s / 9s
LEADER	EPOCH	STATISTICS VERSION	PROCESSES UP	BEACON / TIMEOUT

Processes

Start all | Stop all | Reset demo data

C1 — STANDBY

epoch **2** statistics **v3** N **33**

docs per node **N1=11 N2=11 N3=11**

beacons received **27** from **C2**

last beacon 4s ago (suspects leader at 9s)

`vector [92, 107, 346, 86, 86, 86]`

coordinator — Kill

C2 — LEADER

epoch **2** statistics **v3** N **33**

docs per node **N1=11 N2=11 N3=11**

beacons sent **2**

node beacon replies: N1 ✓ N2 ✓ N3 ✓

`vector [92, 116, 346, 88, 88, 88]`

coordinator — Kill

C3 — stopped

not running

coordinator — Start

N1 — FOLLOWER

epoch **2** statistics **v3** shard docs **11**

beacons received **27** from **C2**

last beacon 4s ago (suspects leader at 9s)

`vector [90, 109, 346, 88, 86, 86]`

node — Kill

N2 — FOLLOWER

epoch **2** statistics **v3** shard docs **11**

beacons received **27** from **C2**

last beacon 4s ago (suspects leader at 9s)

`vector [90, 110, 346, 86, 88, 86]`

node — Kill

N3 — FOLLOWER

epoch **2** statistics **v3** shard docs **11**

beacons received **27** from **C2**

last beacon 4s ago (suspects leader at 9s)

`vector [92, 112, 346, 86, 86, 88]`

node — Kill

Kill a coordinator or node to test failure handling. Vectors are written in the order [C1, C2, C3, N1, N2, N3]. Beacons arrive every 3 s; a process that hears nothing for 9 s suspects the leader.

After killing C3: C3 is stopped, C2 is LEADER at epoch 2, and C1, N1, N2, N3 follow C2. Statistics version and document counts are unchanged.

Before
C3 leader · epoch 1 · 6 / 6 processes up

≈ 9 s of silence
Nodes suspect, coordinators ping, Bully election runs

After
C2 leader · epoch 2 · 5 / 6 up · search and adds keep working

What the nodes noticed first

Lifecycle · Failover · leader of epoch 1 went silent

+0.00s	N2	**suspects the leader (no beacon)** leader: C3 · silent for: 9 [69, 69, 345, 72, 75, 72]
+0.06s	N1	**suspects the leader (no beacon)** leader: C3 · silent for: 9.1 [69, 67, 343, 75, 72, 72]
+0.89s	N3	**suspects the leader (no beacon)** leader: C3 · silent for: 9.9 [69, 69, 346, 72, 72, 75]
+0.00s	N2	**reports the suspected leader** → C1 [69, 69, 345, 72, 76, 72]
+0.06s	N1	**reports the suspected leader** → C1 [69, 67, 343, 76, 72, 72]
+0.89s	N3	**reports the suspected leader** → C1 [69, 69, 346, 72, 72, 76]
+0.00s	N2	**reports the suspected leader** → C2 [69, 69, 345, 72, 77, 72]
+0.03s	C1	**receives the suspicion report** ← N2 reporter: N2 · silent for: 9 [70, 69, 345, 72, 76, 72]
+0.06s	N1	**reports the suspected leader** → C2 [69, 67, 343, 77, 72, 72]

Lifecycle of the failover in the admin console: nodes N1, N2 and N3 each suspect the silent leader after 9 s and report it to the coordinators.

Detection

Every node has its own silence timer. The first to expire sends `ReportSuspect` to all coordinators.

Verification

A coordinator pings C3. When the ping fails it starts the Bully election; other reports arrive while it is busy and are ignored as “already handling”.

Decision

C2 wins, announces epoch 2 and the beacons resume. Each step is an event with a vector clock, so the order is provable.

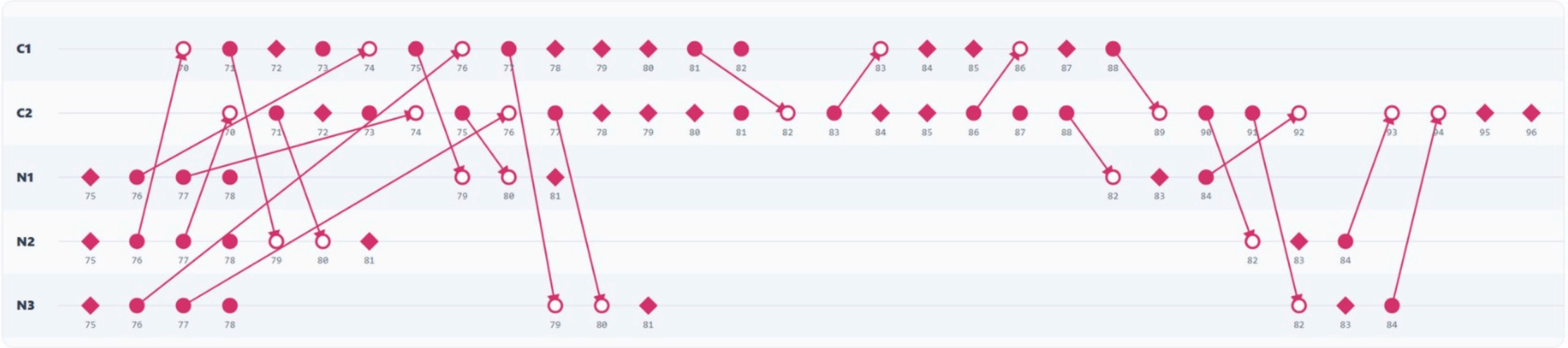
The election as a causal diagram

Vector-clock timeline

20:54:47 Failover · leader of epoch 1 went ▾ | Latest add | Latest election | Latest search | Compare last two adds | Beacons | ▶ Replay | normal ▾ | ☐ show full vectors on the dots

Each dot is an event on one process. Dots show that process's own counter. Arrows are messages: the receiver takes the element-wise maximum of both vectors, then adds one.

● send ○ receive ◆ local event | ● add ● election ● search ● sync ● beacon



Vector-clock timeline of the failover. C3 has no lane because a dead process produces no events. Left: nodes report, coordinators ping and send ELECTION. Right: C2 announces epoch 2 and every process acknowledges.

Left cluster of arrows

N1–N3 → C1, C2 (`ReportSuspect`), then pings and ELECTION messages between coordinators.

Missing lane

C3 does not appear at all: no events, no vector entries beyond what it sent before it died.

Right cluster

C2 → C1, N1, N2, N3: the `Announce` of epoch 2 and the acknowledgements.

What we will show, in order

1	**Start all** Admin console → “Start all”.	Six processes come up, C3 is elected leader (epoch 1) and the beacon counters start rising.
2	**Search** Search page: “asyncio event loop”.	Served by the leader; the node table shows all three shards answering with the same statistics version.
3	**Add a URL** Admin → paste a docs.python.org URL → Add.	Lifecycle: fetch → JSON → ID → least-loaded node → statistics version +1. Search finds it at once.
4	**Two adds at once** “Add 2 sample texts at once”, then “Compare last two adds”.	The two indexing events are concurrent: neither vector dominates.
5	**Kill the leader** Kill C3, watch the countdown bars.	After 9 s nodes report, coordinators ping and elect C2 at epoch 2. Search still works throughout.
6	**Revive C3** Start C3.	It catches up on missed statistics, then bullies back with epoch 3.
7	**Kill a node** Kill N3, add a document, restart N3.	The leader marks N3 down and routes elsewhere; on restart a beacon shows N3 is behind and pushes the missing deltas.

A search engine that now coordinates itself

FA–1 · delivered

- Controlled crawl and logical documents
- Shard-local inverted indexes
- Global TF-IDF and cosine ranking
- Independent gRPC nodes, parallel query fan-out

FA–2 · delivered

- Three coordinators, Bully election with epochs
- Beacon protocol: 3 s interval, 9 s timeout
- Vector clocks on every message, live causal timeline
- Replicated statistics with versions and catch-up
- Live document ingestion and an admin console
- 21 automated tests, 11 of them end-to-end with real failures

Next steps

- Deploy on AWS: EC2 + S3 (designed, not yet built)
- Containers and Kubernetes for nodes and coordinators
- Consensus (for example Raft) for coordinator state
- Replicate shards across nodes

The system is distributed because independent processes own partitions, communicate through RPC, agree on one leader despite failures, and can reason about the order of what happened, while staying useful during partial failure.

Thank you · Questions?

Cloud logos: Wikimedia Commons (see img/CREDITS.txt). All screenshots are of this project's own interface.